Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

Week 6

Hands-on with HBase

Objective 1 - Conceptual Foundations

HBase is a NoSQL database built on top of Hadoop HDFS and modeled after Google's Bigtable. While HDFS and Hive are optimized for batch analytics and large sequential scans, they are not designed for low-latency access. HBase fills this gap by supporting fast, random reads and writes of individual rows, typically in milliseconds. The HBase Master acts as the control plane, managing metadata and assigning regions to RegionServers to balance load, while RegionServers handle client read and write requests. As tables grow, HBase automatically splits them into multiple regions and distributes those regions across different worker nodes in the cluster.

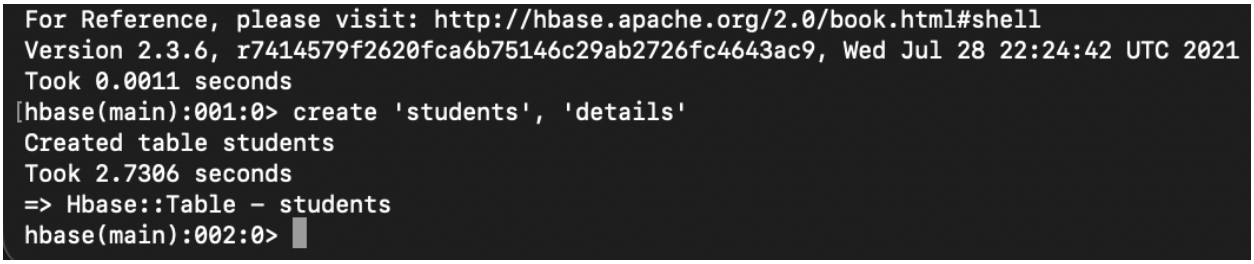
HBase is a wide-column store, where data is organized into tables, rows, and column families. Columns within the same column family are stored together on disk, which makes access patterns critically important to performance. Unlike Hive, which provides an SQL-like interface for analytical queries, HBase is designed for applications that need sub-second lookups across very large datasets. Row key design is central to HBase performance: poorly designed keys can create hotspots where a single RegionServer becomes overloaded. Sequential row keys such as timestamps or auto-

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026
incrementing integers are common anti-patterns, while well-designed keys intentionally spread traffic across regions, often using hashing or salting techniques.

Unlike traditional relational databases, which enforce strong consistency and fixed schemas for transactional workloads, NoSQL systems like HBase relax some relational guarantees to scale horizontally. HBase excels when applications require flexible schemas and real-time access to large volumes of data. It is not intended to replace analytical systems like Hive for long-term data exploration or large aggregations. Instead, HBase is best viewed as an operational data store, optimized for fast point lookups and high-throughput access patterns rather than complex analytical queries.

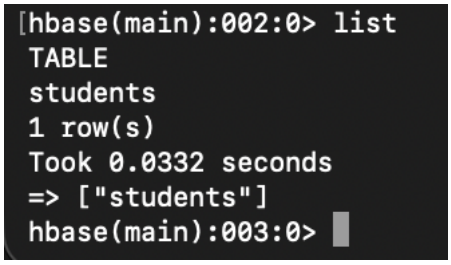
Objective 2 – HBase Table Creation and Management

The students table was created successfully with a single column family named details. The output confirms that HBase accepted the create command and initialized the table without any errors.



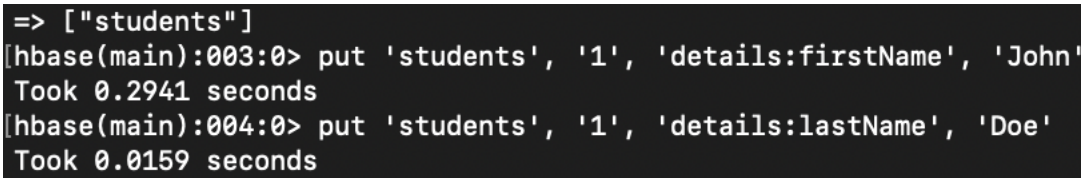
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

The list command displays all tables currently available in HBase. The output confirms that the students table exists and is accessible.

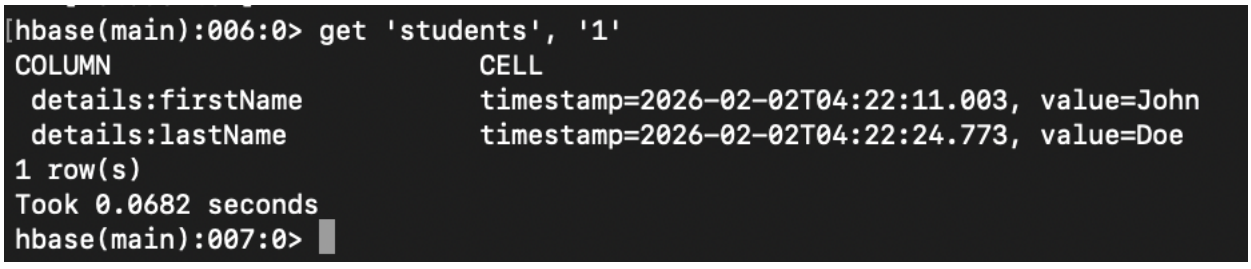


Objective 3 – HBase Data Manipulation

A new row with row key 1 was inserted into the students table by adding firstName and lastName columns under the details column family. This confirms that data can be written to the table successfully using put commands.



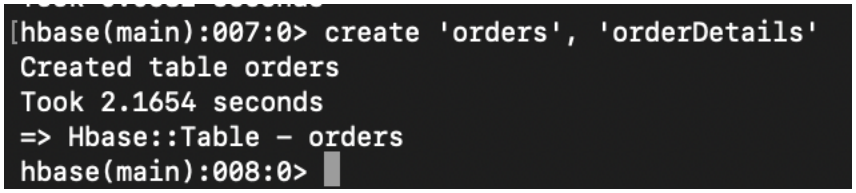
The get command retrieves the row with ID 1 from the students table and displays the stored firstName and lastName values. The output confirms that the data inserted earlier was written correctly and can be read back from HBase.



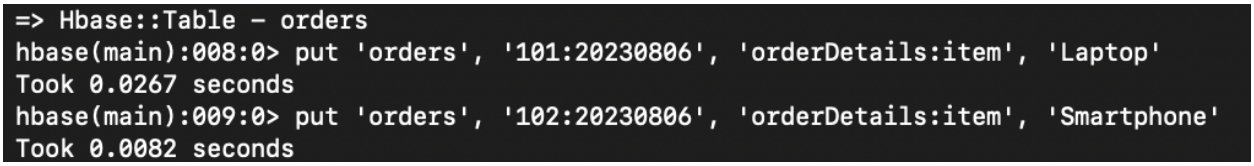
Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

Objective 4 – Composite Row Keys with HBase

A new table named orders was created with a column family called orderDetails. The output confirms that the table was successfully created and is now available for storing order-related data.



Two rows were added to the orders table using composite row keys that include an order ID and date. Each row stores an item value under the orderDetails column family, demonstrating how composite keys can be used to organize and insert order data in HBase.



Objective 5 – Data Generation in HBase

Student records with IDs 2 through 100 were populated in the students table by inserting firstName and lastName values following a consistent naming pattern. This was done to quickly generate sample data for testing and later modification operations.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

```
hbase(main):011:0> (2..100).each do |i|
hbase(main):012:1*    first_name = "Student#{i}"
hbase(main):013:1>    last_name = "LastName#{i}"
hbase(main):014:1>    put 'students', "#{i}", 'details:firstName', first_name
hbase(main):015:1>    put 'students', "#{i}", 'details:lastName', last_name
hbase(main):016:1> end
Took 0.7252 seconds
Took 0.0053 seconds
Took 0.0068 seconds
Took 0.0049 seconds
Took 0.0050 seconds
Took 0.0061 seconds
Took 0.0068 seconds
Took 0.0068 seconds
Took 0.0065 seconds
Took 0.0059 seconds
Took 0.0069 seconds
```

The scan command lists all rows in the students table along with their stored firstName and lastName values. The output confirms that multiple student records were successfully inserted and that the table contains data across a wide range of student IDs.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

```
hbase(main):017:0> scan 'students'
 ROW                           COLUMN+CELL
  1                            column=details:firstName, timestamp=2026-02-02T04:22:11.003, value=John
  1                            column=details:lastName, timestamp=2026-02-02T04:22:24.773, value=Doe
  10                           column=details:firstName, timestamp=2026-02-02T04:26:26.789, value=Student10
  10                           column=details:lastName, timestamp=2026-02-02T04:26:26.796, value=LastName10
  100                          column=details:firstName, timestamp=2026-02-02T04:26:27.695, value=Student100
  100                          column=details:lastName, timestamp=2026-02-02T04:26:27.698, value=LastName100
  11                           column=details:firstName, timestamp=2026-02-02T04:26:26.804, value=Student11
  11                           column=details:lastName, timestamp=2026-02-02T04:26:26.811, value=LastName11
  12                           column=details:firstName, timestamp=2026-02-02T04:26:26.819, value=Student12
  12                           column=details:lastName, timestamp=2026-02-02T04:26:26.825, value=LastName12
  13                           column=details:firstName, timestamp=2026-02-02T04:26:26.832, value=Student13
  13                           column=details:lastName, timestamp=2026-02-02T04:26:26.840, value=LastName13
  14                           column=details:firstName, timestamp=2026-02-02T04:26:26.846, value=Student14
  14                           column=details:lastName, timestamp=2026-02-02T04:26:26.857, value=LastName14
  15                           column=details:firstName, timestamp=2026-02-02T04:26:26.865, value=Student15
  15                           column=details:lastName, timestamp=2026-02-02T04:26:26.871, value=LastName15
  16                           column=details:firstName, timestamp=2026-02-02T04:26:26.877, value=Student16
  16                           column=details:lastName, timestamp=2026-02-02T04:26:26.882, value=LastName16
  17                           column=details:firstName, timestamp=2026-02-02T04:26:26.887, value=Student17
  17                           column=details:lastName, timestamp=2026-02-02T04:26:26.893, value=LastName17
  18                           column=details:firstName, timestamp=2026-02-02T04:26:26.899, value=Student18
  18                           column=details:lastName, timestamp=2026-02-02T04:26:26.906, value=LastName18
  19                           column=details:firstName, timestamp=2026-02-02T04:26:26.911, value=Student19
  19                           column=details:lastName, timestamp=2026-02-02T04:26:26.917, value=LastName19
  2                            column=details:firstName, timestamp=2026-02-02T04:26:26.671, value=Student2
  2                            column=details:lastName, timestamp=2026-02-02T04:26:26.681, value=LastName2
```

Objective 6 – Advanced Data Manipulation in HBase

The loop iterates over student IDs 2 through 50, reads each student's firstName, and replaces the prefix Student with Scholar when present. This updates the firstName values to reflect the new naming convention for that range of students.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

```
Took 0.2215 seconds
hbase(main):018:0> (2..50).each do |id|
hbase(main):019:1*   row = id.to_s
hbase(main):020:1>   value = get 'students', row, 'details:firstName'
hbase(main):021:1>
hbase(main):022:1*   if value && value.include?('Student')
hbase(main):023:2>     new_value = value.sub('Student', 'Scholar')
hbase(main):024:2>     put 'students', row, 'details:firstName', new_value
hbase(main):025:2>   end
hbase(main):026:1> end
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.671, value=Student2
1 row(s)
Took 0.0168 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.687, value=Student3
1 row(s)
Took 0.0053 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.700, value=Student4
1 row(s)
Took 0.0044 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.714, value=Student5
1 row(s)
Took 0.0065 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.731, value=Student6
1 row(s)
Took 0.0062 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.745, value=Student7
1 row(s)
Took 0.0059 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.761, value=Student8
1 row(s)
Took 0.0055 seconds
COLUMN                        CELL
 details:firstName            timestamp=2026-02-02T04:26:26.773, value=Student9
1 row(s)
```

A new middle_name column was added under the details column family for students with IDs 51 through 75, with values following the pattern MidName followed by

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026
the student ID. This was done to demonstrate how HBase allows new columns to be added

dynamically without altering the existing table schema.

```
hbase(main):027:0> (51..75).each do |i|
hbase(main):028:1*   put 'students', i.to_s, 'details:middle_name', "MidName#{i}"
hbase(main):029:1> end
Took 0.0077 seconds
Took 0.0050 seconds
Took 0.0050 seconds
Took 0.0044 seconds
Took 0.0049 seconds
Took 0.0046 seconds
Took 0.0052 seconds
Took 0.0054 seconds
Took 0.0052 seconds
Took 0.0057 seconds
Took 0.0053 seconds
Took 0.0046 seconds
Took 0.0044 seconds
Took 0.0046 seconds
Took 0.0048 seconds
Took 0.0043 seconds
Took 0.0037 seconds
Took 0.0048 seconds
Took 0.0048 seconds
Took 0.0049 seconds
Took 0.0046 seconds
Took 0.0045 seconds
Took 0.0043 seconds
Took 0.0039 seconds
Took 0.0051 seconds
=> 51..75
hbase(main):030:0> get 'students', '51'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.291, value=Student51
 details:lastName                       timestamp=2026-02-02T04:26:27.295, value=LastName51
 details:middle_name                    timestamp=2026-02-02T04:30:37.021, value=MidName51
1 row(s)
Took 0.0105 seconds
hbase(main):031:0> get 'students', '75'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.500, value=Student75
 details:lastName                       timestamp=2026-02-02T04:26:27.503, value=LastName75
 details:middle_name                    timestamp=2026-02-02T04:30:37.157, value=MidName75
1 row(s)
Took 0.0077 seconds
hbase(main):032:0>
```

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

This shows a loop in the HBase shell that iterates over student IDs 76 through 100 and updates each student's last_name by appending the suffix _Modified. For each row, the existing last_name value is read and then written back as a new version with the modified value.

```
hbase(main):032:0> (76..100).each do |i|
hbase(main):033:1*   row = i.to_s
hbase(main):034:1>   value = get 'students', row, 'details:last_name'
hbase(main):035:1>
hbase(main):036:1*   if value
hbase(main):037:2>     put 'students', row, 'details:last_name', "#{value}_Modified"
hbase(main):038:2>   end
hbase(main):039:1> end
COLUMN                                  CELL
0 row(s)
Took 0.0089 seconds
COLUMN                                  CELL
0 row(s)
Took 0.0032 seconds
COLUMN                                  CELL
0 row(s)
Took 0.0042 seconds
COLUMN                                  CELL
0 row(s)
Took 0.0031 seconds
COLUMN                                  CELL
0 row(s)
Took 0.0031 seconds
COLUMN                                  CELL
0 row(s)
Took 0.0035 seconds
COLUMN                                  CELL
```

This screenshot shows student 76 before and after updating the last name, where a new last_name value with the suffix _Modified was added under the details column family.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026
No data was removed; instead, a new version of the last_name column was written, and

the previous value remains as an older version in HBase.

```
hbase(main):040:0> get 'students', '76'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.508, value=Student76
 details:lastName                       timestamp=2026-02-02T04:26:27.513, value=LastName76
1 row(s)
Took 0.0072 seconds
hbase(main):041:0> put 'students', '76', 'details:last_name', 'LastName76_Modified'
Took 0.0053 seconds
hbase(main):042:0> get 'students', '76'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.508, value=Student76
 details:lastName                       timestamp=2026-02-02T04:26:27.513, value=LastName76
 details:last_name                      timestamp=2026-02-02T04:31:57.655, value=LastName76_Modified
1 row(s)
Took 0.0072 seconds
hbase(main):043:0>
```

```
hbase(main):043:0> (90..100).each do |i|
hbase(main):044:1*   deleteall 'students', i.to_s, 'details'
hbase(main):045:1> end
Took 0.0081 seconds
Took 0.0054 seconds
Took 0.0046 seconds
Took 0.0044 seconds
Took 0.0047 seconds
Took 0.0047 seconds
Took 0.0042 seconds
Took 0.0049 seconds
Took 0.0066 seconds
Took 0.0052 seconds
Took 0.0043 seconds
=> 90..100
hbase(main):046:0> get 'students', '90'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.636, value=Student90
 details:lastName                       timestamp=2026-02-02T04:26:27.640, value=LastName90
1 row(s)
Took 0.0069 seconds
hbase(main):047:0> get 'students', '100'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.695, value=Student100
 details:lastName                       timestamp=2026-02-02T04:26:27.698, value=LastName100
1 row(s)
Took 0.0056 seconds
hbase(main):048:0>
```

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
February 1, 2026

This screenshot shows the results of running HBase get commands to retrieve the details column family for students with IDs 40, 60, 80, and 90 after performing the updates. The output verifies that student 60 includes a middle_name column, while the resulting rows display the current first and last name values stored for each student.

```
Took 0.0050 seconds
hbase(main):048:0> get 'students', '40', 'details'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.179, value=Student40
 details:lastName                       timestamp=2026-02-02T04:26:27.184, value=LastName40
1 row(s)
Took 0.0057 seconds
hbase(main):049:0> get 'students', '60', 'details'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.378, value=Student60
 details:lastName                       timestamp=2026-02-02T04:26:27.382, value=LastName60
 details:middle_name                    timestamp=2026-02-02T04:30:37.076, value=MidName60
1 row(s)
Took 0.0058 seconds
hbase(main):050:0> get 'students', '80', 'details'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.545, value=Student80
 details:lastName                       timestamp=2026-02-02T04:26:27.550, value=LastName80
1 row(s)
Took 0.0058 seconds
hbase(main):051:0> get 'students', '90', 'details'
COLUMN                                  CELL
 details:firstName                      timestamp=2026-02-02T04:26:27.636, value=Student90
 details:lastName                       timestamp=2026-02-02T04:26:27.640, value=LastName90
1 row(s)
Took 0.0057 seconds
hbase(main):052:0>
```